

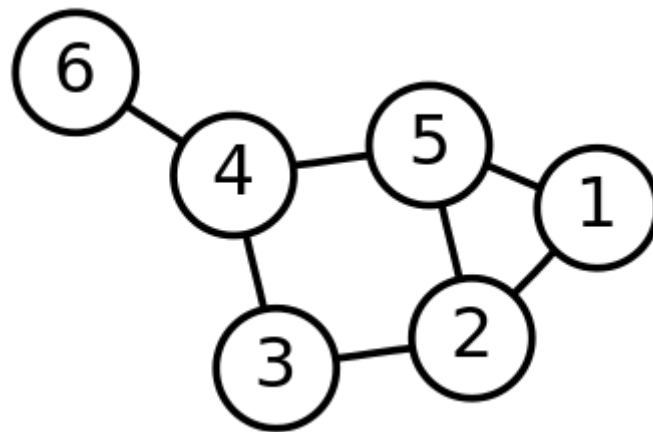
Algoritmi de traversare a grafurilor (DFS & BFS)

1 Obiective

În lucrarea de față vom prezenta cum se pot parcurge grafurile în Prolog. Grafurile le vom reprezenta prin predicate `edge`.

2 Considerații teoretice

Exemplu de graf neorientat:



În Prolog:

Code

```
edge(1,2).  
edge(1,5).  
edge(2,3).  
edge(2,5).  
edge(3,4).  
edge(4,5).  
edge(4,6).
```

```
is_edge(X,Y):- edge(X,Y);edge(Y,X).
```

2.1 Parcurgere în adâncime (DFS)

Ca în lucrarea anterioară, vom folosi forma *edge-clause* de prezentare a grafurilor. Deoarece parcurgerea în adâncime este un mecanism al Prolog-ului, toate predicatele drumurilor din lucrarea anterioară deja folosesc o strategie DFS de căutare. Mai jos aveți predicatul care implementează căutarea DFS dintr-un nod sursă (prin explorarea componentelor conexe a nodului sursă). Această implementare se bazează pe conceptul de **Proces în 2 stagii/pași** discutat în sesiuni anterioare de laborator.

Ne vom folosi de un predicat auxiliar pentru a stoca nodurile deja vizitate.

Code

```
:- dynamic nod_vizitat/1.

% dfs(Graph, Source, Path)
dfs(G,X,_) :- df_search(G,X). % pas1. parcurgerea nodurilor
% când parcurgerea se termină, începe colectarea
dfs(_,_,L) :- !, collect(L). % pas2. colectarea rezultatelor

% predicatul de traversare
df_search(G,X):-
    % salvăm X ca nod vizitat
    assertz(nod_vizitat(X)),
    % luăm un prim edge de la X la Y, restul le vom găsi prin backtracking
    call(G, X, Y),
    % verificăm dacă acest Y a fost deja vizitat
    not(nod_vizitat(Y)),
    % dacă nu a fost - de aceea avem nevoie de negare -
    % atunci vom continua parcurgerea prin mutarea nodului curent la Y
    df_search(G,Y).

% predicatul de colectare - colectarea se face în ordine
collect([X|R]):-
    % scoatem fiecare nod vizitat
    retract(nod_vizitat(X)),!,
    collect(R).
% rezultatul se construiește la întoarcere
collect([]).
```

Urmărește execuția la:

Query

```
?- trace, dfs(is_edge,1,R).
R = [1, 2, 3, 4, 5, 6].
```

2.2 Parcurgerea în lățime (BFS)

Strategia BFS se folosește de o coadă pentru a reține ordinea de expansiune a nodurilor. În fiecare pas, un nou nod este citit din coadă și expandat – toți vecinii nevizitați vor fi adăugați în coadă.

Ne vom folosi de un predicat dinamic pentru a reține nodurile de expandat. Coadă este „implementată” prin modul în care inserăm (*assert*) și extragem (*retract*) acest noduri vizitate. Această implementare se bazează pe conceptul de **Proces în 2 stagii/pași** discutat în sesiuni anterioare de laborator.

```
Code
:- dynamic nod_vizitat/1.
:- dynamic coada/1.      % coada reține nodurile care trebuie expandate

% bfs(Source, Path)
bfs(G,X,_):-             % pas1. parcurgerea nodurilor
    assertz(nod_vizitat(X)),      % adăugăm sursa ca nod vizitat
    assertz(coada(X)),           % adăugăm sursa în coadă
    bf_search(G).
bfs(_,_,R):- !, collect(R).      % pas2. colectarea rezultatelor

bf_search(G):-
    retract(coada(X)),           % scoatem nodul care trebuie expandat
    expand(G,X), !,              % apelăm predicatul de expansiune
    bf_search(G).               % recursivitate

expand(G,X):-
    call(G, X, Y),              % găsim un nod Y cu o muchie la X-ul dat
    not(nod_vizitat(Y)),        % verificăm dacă Y a fost vizitat
    assertz(nod_vizitat(Y)),    % adăugăm Y la nodurile vizitate
    assertz(coada(Y)),          % adăugăm Y în coadă pentru expansiune
                                % la un moment dat
    fail.                       % fail necesar pentru a găsi un alt Y
expand(_,_).
```

Urmărește execuția la:

```
Query
?- trace, bfs(is_edge,1,R).
R = [1, 2, 5, 3, 4, 6].
```

Dar algoritmul BFS poate sa fie implementat și fără utilizarea efectelor laterale. În această implementare vom utiliza reprezentarea de *neighbor/2* pentru graf și vom crea coada (Q) ca o listă și vom reține o listă de noduri vizitate (V).

Code

```
neighbor(1, [2,5]).
neighbor(2, [1,3,5]).
neighbor(3, [2,4]).
neighbor(4, [3,5,6]).
neighbor(5, [1,2,4]).
neighbor(6, [4]).

bfs1(G, X, R) :-
    bfs1(G, [X], [], R).

bfs1(_, [], _, []).
bfs1(G, [X|Q], V, [X|R]) :-
    \+member(X, V), !,
    call(G, X, Ns),      % G(X, Ns) = neighbor(X,Ns)
    remove_visited(Ns, V, RemNs),
    append(Q, RemNs, NewQ),
    bfs1(G, NewQ, [X|V], R).
bfs1(G, [_|Q], V, R) :-
    bfs1(G, Q, V, R).

remove_visited([], _, []).
remove_visited([H|T], V, [H|R]) :- \+member(H, V), !, remove_visited(T, V, R).
remove_visited([_|T], V, R) :- remove_visited(T, V, R).
```

Urmărește execuția la:

Query

```
?- trace, bfs1(neighbor,1,R).
R = [1, 2, 5, 3, 4, 6].
```

2.3 Best-First Search

Algoritmul *Best-first search* este o strategie de căutare greedy, care se folosește de o euristică pentru a estima costul unui drum de la nodul curent la nodul target. În fiecare pas, algoritmul selecționează nodul cu distanța estimată cea mai mică până la nodul target (folosind funcția euristică).

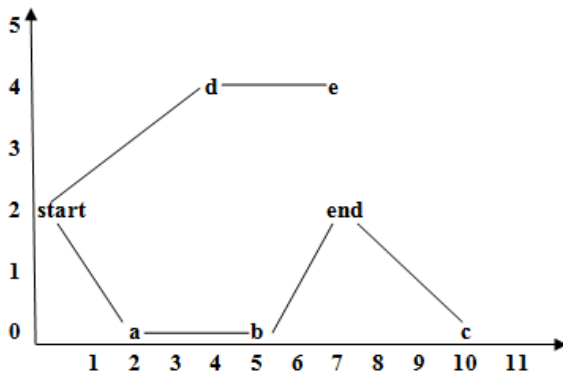


Figura 1: Un exemplu de graf pentru algoritmul best-first search

Graful de mai sus poate fi reprezentat folosind o variație a formei neighbor list-clause:

Code

```
pos_vec(start,0,2,[a,d]).
pos_vec(a,2,0,[start,b]).
pos_vec(b,5,0,[a,c, end]).
pos_vec(c,10,0,[b, end]).
pos_vec(d,3,4,[start,e]).
pos_vec(e,7,4,[d]).
pos_vec(end,7,2,[b,c]).

is_target(end).
```

Nodul de final este specificat ca nodul target, folosind o clauză de predicat. Specificațiile predicatului sunt prezentate mai jos:

Code

```
best([], []):-!.
best([[Target|Rest]|_], [Target|Rest]):- is_target(Target),!.
best([[H|T]|Rest], Best):-
    pos_vec(H,_,_, Neighb),
    expand(Neighb, [H|T], Rest, Exp),
    quick_sort(Exp, SortExp, []),
    best(SortExp, Best).

% Bazat pe calea curentă (al doilea argument), predicatul expand/4
% caută prin vecinii ultimului nod expandat (primul argument)
expand([],_,Exp,Exp):- !.
expand([H|T],Path,Rest,Exp):-
    \+(member(H,Path)), !, expand(T,Path,[[H|Path]|Rest],Exp).
expand([_|T],Path,Rest,Exp):- expand(T,Path,Rest,Exp).
```

% Predicatul quick_sort/3 utilizează liste diferență

```
quick_sort([H|T],S,E):-  
    partition(H,T,A,B),  
    quick_sort(A,S,[H|Y]),  
    quick_sort(B,Y,E).  
quick_sort([],S,S).
```

**% În acest caz, predicatul partition/4 folosește un predicat auxiliar
% order/2 care definește modul de a partiționa ca fiind
% bazat pe distanțe**

```
partition(H,[A|X],[A|Y],Z):- order(A,H),!, partition(H,X,Y,Z).  
partition(H,[A|X],Y,[A|Z]):- partition(H,X,Y,Z).  
partition(_,[],[],[]).
```

% predicat care calculează distanța între două noduri

```
dist(Node1,Node2,Dist):-  
    pos_vec(Node1, X1, Y1, _),  
    pos_vec(Node2, X2, Y2, _),  
    Dist is (X1-X2)*(X1-X2)+(Y1-Y2)*(Y1-Y2).
```

% predicatul order/2 bazat pe distanțe folosit in partition/4

```
order([Node1|_],[Node2|_]):-  
    is_target(Target),  
    dist(Node1,Target,Dist1),  
    dist(Node2,Target,Dist2),  
    Dist1<Dist2.
```

Urmărește execuția la:

Query

?- trace, best([[start]], Best).

3 Exerciții

1. Modificați predicatul DFS astfel încât să caute noduri numai până la o anumită adâncime (DLS – Depth-Limited Search). Setați limita de adâncime printr-un predicat, `depth_max(2)`, de exemplu.

Code

```
edge_ex1(a,b).  
edge_ex1(a,c).  
edge_ex1(b,d).  
edge_ex1(d,e).  
edge_ex1(c,f).  
edge_ex1(e,g).  
edge_ex1(f,h).
```

Query

```
?- dfs(edge_ex1,a,DFS), dls(edge_ex1,a,DLS).  
DFS = [a, b, d, e, g, c, f, h],  
DLS = [a, b, d, c, f].
```

2. Având implementarea algoritmului BFS *fără efecte laterale*, modificați *fără efecte laterale* astfel încât să funcționeze pe reprezentarea de *edge* în loc de reprezentarea de *neighbor*.

Query

```
?- bfs2(is_edge,a, R).  
R = [a, b, c, d, e].
```

3. Scrieți implementarea algoritmului DFS *fără efecte laterale* pentru reprezentările de *edge* și *neighbor*.

Query

```
?- dfs1(neighbor,1,R).  
R = [1, 2, 3, 4, 5, 6].  
  
?- dfs2(is_edge,1,R).  
R = [1, 2, 3, 4, 5, 6].
```